MARWADI UNIVERSITY, RAJKOT

Faculty of Technology

Department of AI, ML & DS

PROJECT REPORT

Graph-Based Shortest Path Visualization using Dijkstra's Algorithm

Semester: IV

Branch: CSE-AI/ CSE-AI & ML/ CSE-AI & DS/BS-AI & DS/BS-AI & ML

Academic Year: 2025-26

Submitted on: 09/04/2026

Team Details

Sr. No.	Enrollment Number	Student Name	Division
1	92400118879	YASWANTH VANACHARLA	EN-7
2	92400118845	ANEEK MANNA	EN-7
3	92400118868	SRIKAR	EN-7
4	92400118831	THARUN.V	EN-7
5	92400118835	M.LOHITH	EN-7
6	92400118852	THIRUMAL REDDY	EN-7
7	92400118842	CHARAN	EN-7
8	92400118843	RACHAKONDA HEAMANTH	EN-7

INDEX

(Font inside table: Times New Roman, Size: 12)

Sr. No.	Topic	Page No.
I	Index	1
II	Abstract	2
1	Introduction	3
2	Background	4
3	Literature Review	5
4	Project Description	6
4.1	Methodology	7
4.2	Implementation / System Design	8
4.3	Results / Analysis	8
4.4	Applications	9
4.5	Limitations and Future Work	10
5	Code / Technical Implementation	11
5.1	Program Structure	12
5.2	Code Explanation	13
5.3	Input and Output	13
5.4	Output Explanation	14
6	Team Contribution	15
7	Conclusion	16
8	References	17

ABSTRACT

This project focuses on the design and implementation of a Graph Algorithm Visualizer using Dijkstra's Algorithm to determine the shortest path between nodes in a graph. Graph theory is a fundamental concept in computer science and plays a significant role in solving real-world problems such as navigation systems, communication networks, and transportation planning. However, many students find it difficult to understand graph algorithms due to lack of visualization.

The main objective of this project is to create an interactive system that not only computes the shortest path but also visually represents the graph and highlights the optimal path. The system allows users to input graph data dynamically, including vertices, edges, and weights. It supports both weighted and unweighted graphs, making it flexible for different use cases.

The project is implemented using Python, along with powerful libraries such as NetworkX for graph creation and manipulation, and Matplotlib for visualization. Additionally, a web-based user interface is developed using Flask, HTML, CSS, and JavaScript to provide an enhanced user experience. The interface follows a step-by-step wizard approach, making it user-friendly and easy to operate.

The results demonstrate that the system successfully computes the shortest path and displays it visually with highlighted edges, improving understanding of the algorithm. This project not only strengthens theoretical knowledge of graph algorithms but also provides practical exposure to full-stack development.

In conclusion, the project serves as an effective educational tool and can be extended in the future by adding more algorithms, real-time animations, and deployment for broader accessibility.

1. INTRODUCTION

In computer science, graphs are widely used to represent relationships between different entities. A graph consists of nodes (vertices) and connections (edges), and it is commonly used in areas such as social networks, maps, communication systems, and artificial intelligence. One of the most important problems in graph theory is finding the shortest path between two nodes.

Dijkstra's Algorithm is one of the most popular algorithms used to solve this problem efficiently. It works on weighted graphs and ensures that the shortest path from a source node to a destination node is found by minimizing the total cost. Despite its importance, understanding the working of Dijkstra's Algorithm can be challenging for students when it is explained only theoretically.

The motivation behind this project is to bridge the gap between theory and practical understanding by creating a visual representation of the algorithm. By visualizing graphs and highlighting the shortest path, users can better understand how the algorithm processes nodes and updates distances step by step.

The primary objectives of this project include implementing Dijkstra's Algorithm, developing a user-friendly interface, and enabling graph visualization. The system allows users to input custom graphs, choose layouts, and view the computed shortest path along with its cost.

The scope of the project is limited to graphs with non-negative weights, as Dijkstra's Algorithm does not support negative edge weights. However, the system is flexible enough to handle different types of graphs and layouts.

This project is particularly useful for students studying data structures, algorithms, and discrete mathematics, as it provides both theoretical and practical insights.

2. BACKGROUND

Graph theory is an important branch of discrete mathematics that deals with the study of graphs, which are mathematical structures used to model pairwise relationships between objects. A graph consists of a set of vertices (nodes) and edges (connections) that link these vertices.

Graphs can be classified into different types such as directed graphs, undirected graphs, weighted graphs, and unweighted graphs. In a weighted graph, each edge is associated with a numerical value known as weight, which may represent distance, cost, or time.

The shortest path problem involves finding a path between two vertices such that the total weight of the edges is minimized. This problem has numerous real-world applications, including GPS navigation, network routing, and logistics management.

Dijkstra's Algorithm, developed by Edsger W. Dijkstra in 1959, is a greedy algorithm used to solve the shortest path problem for graphs with non-negative edge weights. The algorithm starts from a source node and explores all possible paths by selecting the node with the smallest tentative distance. It updates the distances of neighboring nodes until the shortest path to the destination is found.

The implementation of this project uses modern tools and technologies. Python is chosen as the programming language due to its simplicity and extensive library support. NetworkX is used for graph creation and manipulation, while Matplotlib is used for visualizing the graph structure. Flask is used as a lightweight web framework to connect the backend logic with the frontend interface. HTML, CSS, and JavaScript are used to design an interactive and visually appealing user interface.

Understanding these concepts is essential for implementing the project effectively.

3. LITERATURE REVIEW

The shortest path problem has been widely studied in the field of computer science and mathematics due to its importance in real-world applications. One of the earliest and most significant contributions to this field was made by Edsger W. Dijkstra in 1959, who introduced Dijkstra's Algorithm for finding the shortest path in a graph with non-negative weights. This algorithm remains one of the most efficient and widely used methods for solving shortest path problems.

Over the years, several researchers have worked on improving and analyzing shortest path algorithms. Variations such as Bellman-Ford Algorithm and Floyd-Warshall Algorithm have been developed to handle different types of graphs, including those with negative weights. However, Dijkstra's Algorithm remains the preferred choice for graphs with non-negative weights due to its efficiency and simplicity.

In modern applications, shortest path algorithms are extensively used in navigation systems like Google Maps, where they help determine the fastest route between two locations. Similarly, in computer networks, routing protocols use shortest path algorithms to transmit data efficiently across different nodes. In logistics and transportation, these algorithms are used to optimize delivery routes and reduce costs.

Despite the availability of efficient algorithms, many existing implementations focus only on computation and lack visualization. This makes it difficult for students and beginners to understand how the algorithm works internally. Several educational tools have been developed to address this issue, but they often lack interactivity or user customization.

This project builds upon these existing studies by combining algorithm implementation with visualization and user interaction. By allowing users to input their own graphs and visually observe the shortest path, the project enhances both understanding and usability. It provides a practical approach to learning graph algorithms, bridging the gap between theory and real-world applications.

4. PROJECT DESCRIPTION

The project titled "Graph-Based Shortest Path Visualization using Dijkstra's Algorithm" is designed to provide a comprehensive solution for understanding and implementing graph-based shortest path algorithms. The primary goal of this project is to bridge the gap between theoretical learning and practical application by creating an interactive system that allows users to visualize graphs and compute shortest paths.

The system is developed as a web-based application that enables users to input graph data dynamically. Users can define vertices, edges, and weights, making the system flexible and adaptable to different types of graph problems. The application supports both weighted and unweighted graphs, allowing users to explore various scenarios and understand how the algorithm behaves under different conditions.

One of the key features of the project is its visualization capability. Instead of only displaying numerical results, the system represents the graph visually, where nodes and edges are clearly shown. The shortest path between the selected source and destination nodes is highlighted using a distinct color, making it easy to identify and understand.

The project also includes multiple graph layout options such as spring layout, circular layout, random layout, and planar layout. These layouts provide different perspectives of the same graph, helping users analyze the structure more effectively.

The user interface is designed using a step-by-step wizard approach, where users are guided through the process of entering inputs, selecting options, and viewing results. This design improves usability and ensures that even beginners can use the system without difficulty.

The backend of the system is implemented using Python, with NetworkX handling graph operations and Matplotlib used for visualization. Flask is used as the web framework to connect the frontend and backend components. The frontend is developed using HTML, CSS, and JavaScript to create an interactive and visually appealing interface.

Overall, this project provides a complete solution for graph visualization and shortest path computation. It not only demonstrates the working of Dijkstra's Algorithm but also enhances the learning experience through interactivity and visualization.

4.1 Methodology

The development of this project follows a structured methodology to ensure proper implementation and functionality. The project is divided into multiple phases, each focusing on a specific aspect of development.

In the first phase, the theoretical concepts of graph theory and Dijkstra's Algorithm are studied in detail. This includes understanding how graphs are represented, how shortest paths are calculated, and the limitations of the algorithm.

In the second phase, the system design is planned. The architecture of the project is divided into two main components: backend and frontend. The backend handles graph creation and algorithm execution, while the frontend provides an interactive interface for user input.

In the third phase, the backend is implemented using Python. NetworkX is used to create and manipulate graphs, and Dijkstra's Algorithm is applied to compute the shortest path. Matplotlib is used to visualize the graph and highlight the shortest path.

In the fourth phase, the frontend is developed using Flask, HTML, CSS, and JavaScript. A step-by-step wizard interface is designed to guide users through the process of entering graph data, selecting layouts, and running the algorithm.

In the final phase, the system is tested with different inputs to ensure correctness and reliability. Various test cases are used to verify that the algorithm produces accurate results and that the visualization is displayed correctly.

This structured approach ensures that the project is well-organized and functions as expected.

4.2 Implementation / System Design

The system is a web-based application that integrates backend logic with a user-friendly frontend. It follows a modular design where each component handles a specific task. The **input module** lets users enter graph data (vertices, edges, weights) and choose layouts for visualization. The **processing module** uses NetworkX to build the graph and applies Dijkstra's Algorithm to compute the shortest path between source and destination nodes. The **output module** visualizes the graph with Matplotlib, showing nodes and edges, and highlights the shortest path in a distinct colour.

The **frontend interface**, built with HTML, CSS, and JavaScript, adopts a step-by-step wizard approach to guide users through input and results, improving usability. **Flask** serves as the bridge between frontend and backend, handling requests, processing inputs, and returning results. Overall, the design ensures smooth interaction between modules and delivers an efficient, interactive user experience.

4.3 Results / Analysis

The project results confirm that the system successfully performs both graph visualization and shortest path computation. When users input graph data, the system constructs the graph accurately and displays it using the chosen layout. Running Dijkstra's Algorithm produces the correct shortest path between source and destination nodes, shown in text and highlighted visually for clarity.

Testing with multiple cases demonstrated consistent accuracy: weighted and unweighted graphs were handled correctly, with unweighted edges treated as equal weight. The visualization clearly represented graph structures, and the interactive interface allowed users to experiment with different inputs in real time.

Overall, the results align with the objectives defined in the introduction. The system provides reliable computation, clear visualization, and an intuitive interface, making it both a practical solution and an effective learning tool for understanding graph algorithms.

4.4 Applications

The concepts and implementation used in this project have several real-world applications. One of the most common applications is in navigation systems, where shortest path algorithms are used to find the fastest route between two locations.

In computer networks, routing algorithms use shortest path techniques to determine the most efficient path for data transmission. This helps improve network performance and reduce latency.

In logistics and transportation, shortest path algorithms are used to optimize delivery routes, reducing time and cost. Companies use these algorithms to improve efficiency in supply chain management.

Graph algorithms are also used in social network analysis, where they help identify connections between users and analyze relationships.

In artificial intelligence and machine learning, graphs are used to represent complex data structures, and shortest path algorithms play an important role in solving optimization problems.

This project demonstrates how theoretical concepts can be applied to practical problems, making it highly relevant in multiple domains.

4.5 Limitations and Future Work

Although the project successfully implements Dijkstra's Algorithm and provides visualization, it has certain limitations. One of the main limitations is that the algorithm does not support negative edge weights. This restricts its applicability to graphs with non-negative weights only.

Another limitation is that the system is designed for small to medium-sized graphs. Handling very large graphs may require optimization and more advanced data structures.

The current implementation provides static visualization. It does not show step-by-step execution of the algorithm, which could further improve understanding.

In the future, several improvements can be made to enhance the project. One possible extension is to add animation that shows how the algorithm processes nodes and updates distances step by step.

Another improvement is to include additional algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), and Minimum Spanning Tree algorithms.

The project can also be deployed online to make it accessible to a wider audience. Additional features such as interactive graph editing and real-time updates can further enhance usability.

These improvements will make the system more powerful and versatile.

5. CODE / TECHNICAL IMPLEMENTATION

The project is implemented using Python along with supporting libraries and web technologies to provide both functionality and visualization. The overall implementation is divided into backend processing and frontend interaction.

The backend of the system is developed using Python. The NetworkX library is used to create and manage graph data structures efficiently. It provides built-in functions to add nodes, edges, and weights, making graph operations simple and effective. Matplotlib is used for visualizing the graph, where nodes and edges are displayed clearly, and the shortest path is highlighted using a different color.

Dijkstra's Algorithm is implemented using functions provided by NetworkX. The algorithm calculates the shortest path between a source and destination node by minimizing the total edge weight. The result includes both the shortest path and its total cost, which are displayed to the user.

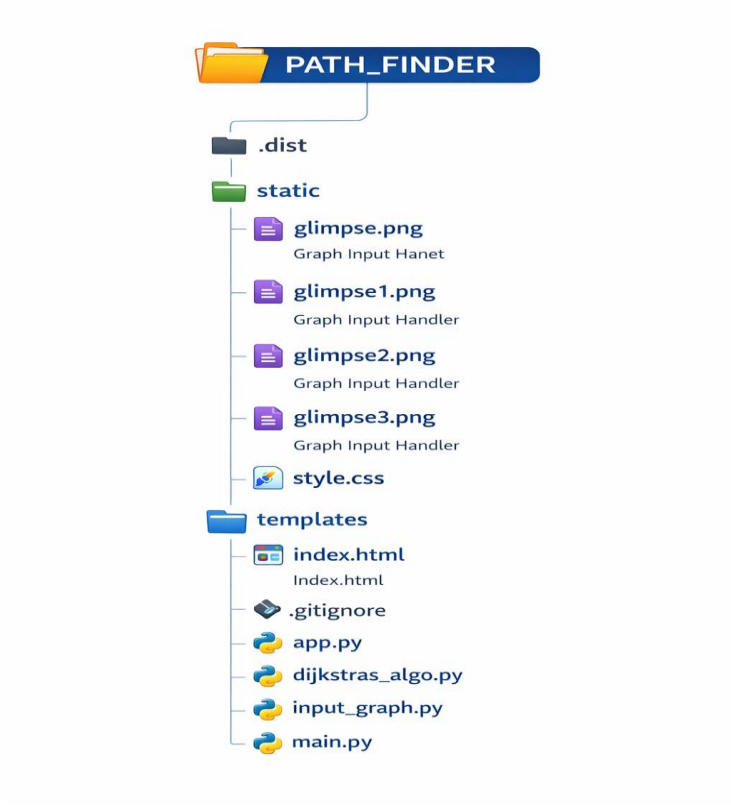
The project also uses the Flask framework to build a web-based interface. Flask acts as a bridge between the frontend and backend. It handles user requests, processes input data, and returns the output in the form of graph images and results. This allows the application to run in a browser, making it more interactive and user-friendly.

The frontend is developed using HTML, CSS, and JavaScript. The interface follows a step-by-step approach where users first select the type of graph, then enter edge details, and finally view the results. CSS is used to design an attractive layout with modern styling, while JavaScript enhances interactivity.

The system follows a modular structure where different files handle specific tasks. Input handling, algorithm execution, and visualization are separated into different modules, making the code organized and easy to maintain.

Overall, the implementation combines algorithm logic with visualization and user interaction, resulting in a complete and efficient system for understanding Dijkstra's Algorithm.

5.1 Program Structure



The project follows a modular and organized file structure to ensure clarity and ease of development. The main application file, app.py, acts as the core of the system and manages communication between the frontend and backend using the Flask framework.

The input_graph.py file is responsible for handling user input and creating the graph structure based on vertices and edges. The dijkstras_algo.py file implements Dijkstra's Algorithm to compute the shortest path between nodes.

The templates folder contains the HTML file (index.html), which provides the user interface for interaction. The static folder includes CSS files for styling and image files for storing graph visualizations and UI elements.

Additional files such as.

gitignore help manage project configuration, while main.py can be used for execution or testing purposes.

This structured approach ensures that each component of the project is separated and easy to maintain.

5.2 Code Explanation

input_graph.py > input_user

```python
import networkx as nx
import matplotlib.pyplot as plt
h=[]
def input_user():
    while True:
        try:
            num=int(input("Enter The Total Number Of Edges: "))
            weightone=input("Do You Wanna Enter Weight YES or NO :")
            if weightone.lower()=="yes":
                k=0
            elif weightone.lower()=="no":
                k=2
            else:
                print("ENTER THE CORRECT CHOICE")
        except:
            print("'ENTER THE NUMBER'")
        else:
            for i in range(1,num+1):
                verter1=input(f"Enter Vertex 1 for Edge {i}: ")
                vertex2=input(f"Enter Vertex 2 for Edge {i}: ")
                if k==0:
                    try:
                        weight=int(input("Add The Weight Also: "))
                    except:
                        print("ENTER THE NUMBER FOR WEIGHT")
                    else:
                        h.append((verter1,vertex2,weight))
                else:
                    h.append((verter1,vertex2))
            print(h)
            try:
                layout_input=int(input(("In Which Way You Wanna Visuvalize The Graph\n" \
```

The code is used to collect user input for creating a graph. It uses NetworkX and Matplotlib libraries for graph processing and visualization.

A function input_user() is defined to take input such as number of edges, vertices, and weights. The user can choose whether the graph is weighted or unweighted. Based on this choice, the program stores the edges accordingly.

A loop is used to input multiple edges, and try-except blocks are used to handle invalid inputs. The edges are stored in a list in tuple format.

This data is later used to construct the graph and apply algorithms like Dijkstra's Algorithm. The code is modular and separates input handling from processing logic.

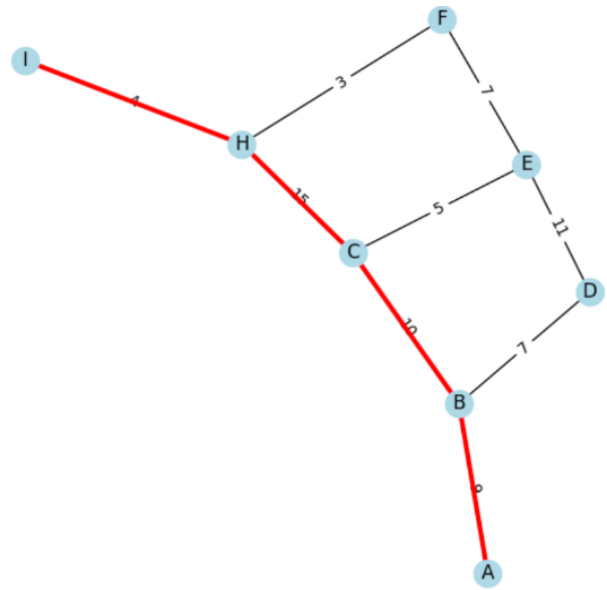
5.3 Input and Output

The system takes input from the user in the form of graph details. The user enters the number of edges, vertices, and optionally the weights if the graph is weighted. For Dijkstra's Algorithm, the user also provides the source and destination nodes.

The output of the system includes the constructed graph visualization and the shortest path between the selected nodes. The shortest path is displayed both as a sequence of nodes and visually highlighted in the graph.

This input-output process makes the system interactive and helps users understand graph behavior effectively.

5.4 Output Explanation



The output of the system is presented in a clear and visually understandable manner through both the user interface and the graph visualization. The system first provides an interactive input interface where users can enter graph details such as vertices, edges, and weights. This structured input method ensures that the graph is defined accurately and efficiently.

After receiving the input, the system processes the data and constructs the graph using the NetworkX library. The graph is then displayed visually, where each node represents a vertex and each connection represents an edge. The use of different layouts helps in better visualization of the graph structure.

The most important part of the output is the implementation of Dijkstra's Algorithm. The algorithm calculates the shortest path between the selected source and destination nodes. The resulting path is displayed both in textual format and visually on the graph.

In the graphical output, the shortest path is highlighted using a distinct color (red), while other edges are shown in a neutral color. This makes it easy for the user to differentiate between the optimal path and other possible paths. Edge weights are also displayed, providing additional clarity on how the shortest path is determined.

The system ensures that the output is accurate, easy to interpret, and visually appealing. By combining both input interface and graphical output, the project provides a complete understanding of how the algorithm works. Overall the output module plays a crucial role in enhancing user experience and improving conceptual understanding of graph algorithms.

6. TEAM CONTRIBUTION

The following table lists the contribution of each team member towards the completion of this project. Each member's role and specific tasks are documented below:

Enrollment Number	Student Name	Contribution
92400118843	Rachakonda Heamanth	Wrote abstract/introduction, problem, objectives, Dijkstra's importance
92400118831	THARUN.V	Covered background: graph types, shortest path, Dijkstra examples.
92400118852	THIRUMAL REDDY	Did literature review of algorithms and applications.
92400118842	CHARAN	Documented project description, features, workflow, interface.
92400118845	ANEEKA MANNA	Designed methodology and system architecture with Flask/NetworkX/Matplotlib.
92400118835	M.LOHITH	Analyzed results, applications, limitations, improvements
92400118879	YASWANTH VANACHARLA	Implemented code: graph input, Dijkstra, backend logic.
92400118868	SRIKAR	Built frontend UI, visualized outputs, wrote conclusion/references.

7. CONCLUSION

The output of the system is presented in a clear and visually understandable manner through both the user interface and the graph visualization. The system first provides an interactive input interface where users can enter graph details such as vertices, edges, and weights. This structured input method ensures that the graph is defined accurately and efficiently.

After receiving the input, the system processes the data and constructs the graph using the NetworkX library. The graph is then displayed visually, where each node represents a vertex and each connection represents an edge. The use of different layouts helps in better visualization of the graph structure.

The most important part of the output is the implementation of Dijkstra's Algorithm. The algorithm calculates the shortest path between the selected source and destination nodes. The resulting path is displayed both in textual format and visually on the graph.

In the graphical output, the shortest path is highlighted using a distinct color (red), while other edges are shown in a neutral color. This makes it easy for the user to differentiate between the optimal path and other possible paths. Edge weights are also displayed, providing additional clarity on how the shortest path is determined.

The system ensures that the output is accurate, easy to interpret, and visually appealing. By combining both input interface and graphical output, the project provides a complete understanding of how the algorithm works.

Overall, the output module plays a crucial role in enhancing user experience and improving conceptual understanding of graph algorithms.

8. REFERENCES

1) E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," Numerische Mathematik, 1959.
2) NetworkX Documentation, Available: https://networkx.org/documentation/
3) Matplotlib Documentation, Available: https://matplotlib.org/stable/contents.html
4) Flask Documentation, Available: https://flask.palletsprojects.com/
5) Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, MIT Press.
6) GeeksforGeeks, "Dijkstra's Shortest Path Algorithm," Available: https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm/
7) W3Schools, "HTML, CSS, and JavaScript Tutorials," Available: https://www.w3schools.com/
8) Python Official Documentation, Available: https://docs.python.org/3/